

Chapter 19: Time and Space Complexity

Personal Notes

Until now you've learned variables (store data), loops (repeat logic), arrays (organize data), and methods (structure logic). Now comes a critical shift in thinking.

It is not enough to just write code that WORKS. From this chapter onward, you'll start thinking about whether your code is EFFICIENT — fast enough and light enough on memory to handle real-world inputs. This is the mindset that separates a beginner from a problem solver.

1. Why Complexity Matters

Imagine you wrote two solutions to the same problem:

- Solution A runs in 1 second
- Solution B runs in 10 seconds

Both give the correct output. Which is better? Obviously Solution A.

The Scale Problem

Imagine a system with users:

1 user	→ 10 operations	→ fine
100 users	→ 1,000 operations	→ fine
1,000 users	→ 100,000 operations	→ still ok
100,000 users	→ 10,000,000,000 ops	→ CRASHES

Efficiency starts mattering exactly when scale increases.

In a small program with 10 inputs, even bad code works. But in a real app with millions of users, the difference between $O(n)$ and $O(n^2)$ is the difference between "runs in 1 second" and "runs out of memory."

2. What is Time Complexity?

Time Complexity measures how the NUMBER OF OPERATIONS grows as the input size grows. That's it. It is a way to describe the SHAPE of how slow your code gets when the input gets bigger.

Important clarification: Time Complexity does NOT measure actual time in seconds. Actual time depends on the machine, the language, the load, and a hundred other things. Time Complexity is hardware-independent — it only describes how operations GROW with input.

Two Programs With Different Growth

Input n	Program A (n)	Program B (n^2)
10	10	100
100	100	10,000
1000	1,000	1,000,000
10000	10,000	100,000,000

A grows linearly. B explodes. Same code style – very different fate.

3. What is Space Complexity?

Space Complexity measures how much EXTRA memory your program uses as the input grows.

Just like time, this also grows with input — and you should care about it for the same reason. A program that's fast but eats 10 GB of memory won't run on a normal laptop.

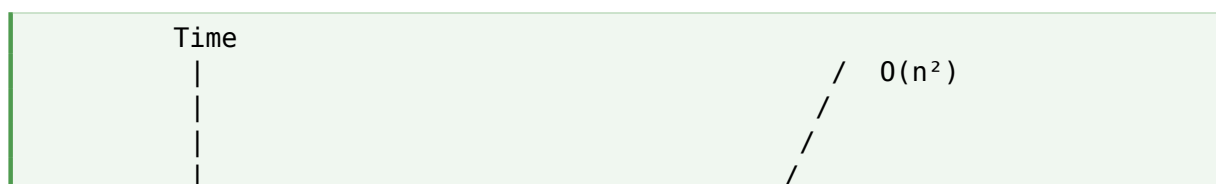
4. Big-O Notation

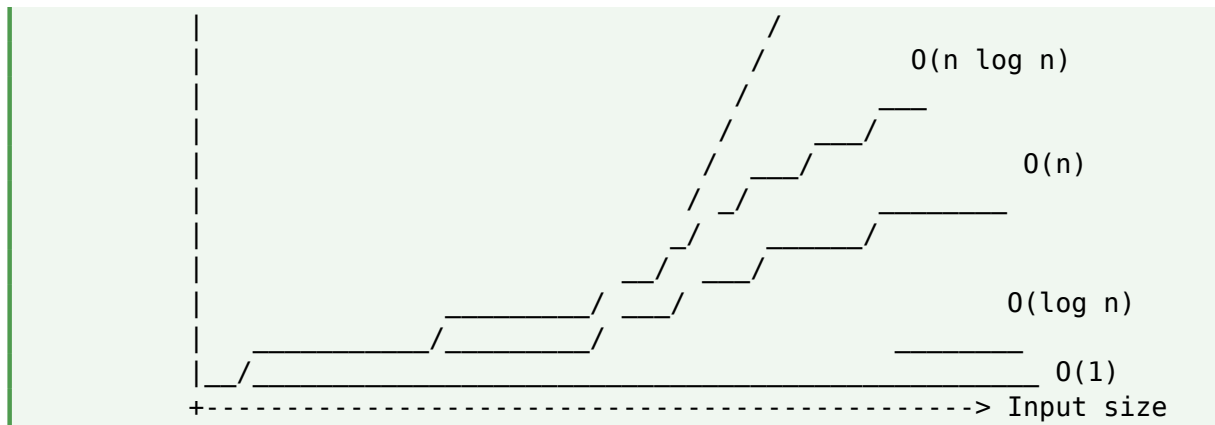
We represent complexity using Big-O notation. Big-O describes the UPPER BOUND of growth — the worst case as input gets large.

The Common Complexities

Notation	Name	Meaning
$O(1)$	Constant	Always the same number of ops
$O(\log n)$	Logarithmic	Cuts the problem in half each step
$O(n)$	Linear	Grows directly with input size
$O(n \log n)$	Linearithmic	Linear $\times$ Log — typical for efficient sorting
$O(n^2)$	Quadratic	Two nested loops over n
$O(n^3)$	Cubic	Three nested loops over n
$O(2^n)$	Exponential	Doubles with each step — avoid for large n
$O(n!)$	Factorial	Disaster for anything beyond tiny n

Visual Growth Comparison





5. The Common Complexities — In Detail

5.1 $O(1)$ — Constant Time

```
int x = 10;
System.out.println(x);

// or accessing an array element:
int firstElement = arr[0];
```

No matter the input size, the number of operations stays the same. Always one step. This is the gold standard — but it's only possible when you're not iterating over anything.

5.2 $O(n)$ — Linear Time

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

Operations grow directly with input. If $n = 100$, you do 100 operations. If $n = 1000$, you do 1000.

```
n = 5    → 5 ops
n = 50   → 50 ops
n = 500  → 500 ops
```

5.3 $O(n^2)$ — Quadratic Time

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + " " + j);
    }
}
```

Outer loop runs n times, inner loop runs n times. Total operations = $n \times n = n^2$.

Dry Run for $n = 3$

```
(0,0) (0,1) (0,2)
```

```
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

Total = 9 = 3^2

5.4 $O(\log n)$ — Logarithmic Time

```
while (n > 1) {
    n = n / 2;
}
```

Each step cuts n in half. That's how binary search works.

```
n = 16 → 8 → 4 → 2 → 1
      step1 step2 step3 step4
```

Number of steps = $\log_2(16) = 4$

For $n = 1024 \rightarrow$ only 10 steps!

For $n = 1,000,000 \rightarrow$ only 20 steps!

5.5 $O(n \log n)$ — Linearithmic Time

You get this when you do something linear (n) inside something logarithmic, like efficient sorting algorithms (Merge Sort, Heap Sort, Quick Sort average case). It's the typical complexity of "good" sorting.

6. How to Calculate Time Complexity — The 5 Rules

Rule 1: Ignore Constants

```
for (int i = 0; i < 2 * n; i++) {
    System.out.println(i);
}
```

This runs $2n$ times. But $2n$ is still linear growth — $O(2n)$ is just $O(n)$. Constants don't change the shape.

```
O(2n)    → O(n)
O(n/2)   → O(n)
O(100n)  → O(n)
O(5)     → O(1)
```

Rule 2: Drop Lower-Order Terms

```
O(n2 + n + 100) → O(n2)
O(n3 + n2)      → O(n3)
O(n + log n)     → O(n)
```

As n gets large, only the highest-order term matters. n^2 dwarfs n . n^3 dwarfs n^2 . Only the biggest term survives.

Rule 3: Nested Loops Multiply

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        // work  
    }  
}
```

Inner loop runs for every iteration of the outer. Total = $n \times n = n^2$. If three levels deep over n , it's n^3 .

Rule 4: Sequential Loops Add

```
for (int i = 0; i < n; i++) { ... }    // n operations  
for (int j = 0; j < n; j++) { ... }    // n more operations  
  
// Total:  $n + n = 2n \rightarrow O(n)$ 
```

Loops that run ONE AFTER ANOTHER (not inside each other) add up. $n + n = 2n = O(n)$. Multiplication only happens for NESTED loops.

Rule 5: Condition-Based Loops

```
for (int i = 1; i < n; i *= 2) { ... }    //  $O(\log n)$   
for (int i = n; i > 0; i /= 2) { ... }    //  $O(\log n)$   
for (int i = 1; i < n; i *= 3) { ... }    //  $O(\log n)$   
for (int i = 0; i < n; i += 2) { ... }    //  $O(n)$ 
```

When the loop variable is MULTIPLIED or DIVIDED each step, you get $O(\log n)$. When ADDED or SUBTRACTED by a constant, you stay at $O(n)$.

7. Worked Problems

Problem 1: Simple Loop

```
for (int i = 0; i < n; i++) {  
    System.out.println(i);  
}
```

Loop runs n times. ✓ Time Complexity = $O(n)$

Problem 2: Nested Loop

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.println(i + j);  
    }  
}
```

$n \times n = n^2$ operations. ✓ Time Complexity = $O(n^2)$

Problem 3: Half Loop

```
for (int i = 0; i < n / 2; i++) {  
    System.out.println(i);  
}
```

Runs $n/2$ times. Drop the constant. ✓ Time Complexity = $O(n)$

Problem 4: Skip by 2

```
for (int i = 0; i < n; i += 2) {  
    System.out.println(i);  
}
```

Runs $n/2$ times. Drop the constant. ✓ Time Complexity = $O(n)$

Problem 5: Logarithmic Loop

```
for (int i = 1; i < n; i *= 2) {  
    System.out.println(i);  
}
```

Goes $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow n$. That's $\log_2(n)$ steps. ✓ Time Complexity = $O(\log n)$

8. Best Case vs Worst Case

The same algorithm can behave differently depending on input. Linear search is the classic example.

```
public static int linearSearch(int[] arr, int target) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == target) {  
            return i;  
        }  
    }  
    return -1;  
}
```

Case	When it happens	Complexity
Best case	Target is at index 0	$O(1)$
Average case	Target is somewhere in the middle	$O(n)$
Worst case	Target is last element, or not present	$O(n)$

Default convention: When people say "the complexity of X is $O(n)$," they usually mean the WORST case. That's the safest thing to plan for — your code must handle the worst input, not just the lucky ones.

9. Space Complexity

Space Complexity is the extra memory your program uses, as a function of input size.

9.1 Examples

$O(1)$ — Constant Space

```
int a = 10;
int b = 20;
int sum = a + b;
```

Three variables, no matter what n is. Always the same amount of memory.

$O(n)$ — Linear Space

```
int[] arr = new int[n];
```

Memory grows with input — an array of size n uses n integers worth of memory.

$O(n^2)$ — Quadratic Space

```
int[][] matrix = new int[n][n];
```

An $n \times n$ matrix uses n^2 cells of memory.

9.2 Auxiliary vs Total Space

Two terms often used:

- Input Space — memory used by the input itself (the array you were given)
- Auxiliary Space — EXTRA memory your algorithm uses beyond the input

```
public static int findSum(int[] arr) {
    int sum = 0;                // 0(1) auxiliary
    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];
    }
    return sum;
}

// Input space:      0(n) — the array itself
// Auxiliary space:  0(1) — just one variable
```

When people say "space complexity," they usually mean auxiliary space. Don't double-count the input.

10. The Time-Space Tradeoff

In real-world code, you often have to choose between using more time or more memory. These two are linked.

More space $\longleftrightarrow$ More time

Brute-force solution:

- Less memory needed
- More operations
- Slower for large input

Optimized solution (with extra data structures):

- Uses extra memory (hash map, cache, etc.)
- Far fewer operations
- Faster for large input

Classic example: searching. A linear scan is $O(n)$ time and $O(1)$ extra space. Using a HashSet first is $O(1)$ lookup time but $O(n)$ extra space. The right choice depends on the situation.

11. Common Mistakes

Mistake 1: Treating time complexity as actual seconds

$O(n)$ does not mean "runs in n seconds." It means the number of operations grows linearly. Real time depends on the machine, language, and many other factors.

Mistake 2: Not removing constants

```
// All of these are  $O(n)$ , not  $O(2n)$  or  $O(100n)$ :  
for (int i = 0; i < 2 * n; i++) { ... }  
for (int i = 0; i < n + 5; i++) { ... }  
for (int i = 0; i < 100 * n; i++) { ... }
```

Mistake 3: Confusing nested loops with sequential

```
// Nested = multiply →  $O(n^2)$   
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++) { ... }  
  
// Sequential = add →  $O(n + n) = O(n)$   
for (int i = 0; i < n; i++) { ... }  
for (int j = 0; j < n; j++) { ... }
```

Mistake 4: Forgetting that $O(\log n)$ is very fast

Even for $n = 1$ billion, $\log_2(n)$ is only about 30. Logarithmic algorithms are dramatically faster than linear ones at scale.

Mistake 5: Ignoring hidden complexity in library calls

Calling `list.contains(x)` inside a loop looks innocent — but `contains()` is $O(n)$ by itself, so the whole loop becomes $O(n^2)$.

Mistake 6: Forgetting input vs auxiliary space

If you're given an array of size n , you don't count that as YOUR algorithm's space — it's the input. Only count what YOU add on top.

12. Why This Matters for Interviews

Time and Space Complexity is asked in EVERY serious coding interview. Specifically:

- After you solve a problem, they ALWAYS ask: "What's the time and space complexity?"
- They use it to compare your solution against the optimal one
- It signals whether you can think about scale, not just correctness
- A working $O(n^2)$ solution often loses to an $O(n \log n)$ one — even if both are correct

Being able to instantly state the complexity of your code is non-negotiable for placements.

13. Practice Problems (with Answers)

Try to solve each before looking at the answer. The answers are based on the rules from Section 6.

Basic Level

Q1

```
for (int i = 0; i < n; i++) { ... }
```

Answer: $O(n)$ — simple linear loop

Q2

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) { ... }  
}
```

Answer: $O(n^2)$ — inner loop runs $0 + 1 + 2 + \dots + (n-1) = n(n-1)/2$ times $\rightarrow O(n^2)$

Q3

```
for (int i = n; i > 0; i /= 2) { ... }
```

Answer: $O(\log n)$ — variable is halved each step

Q4

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < m; j++) { ... }
```

Answer: $O(n \times m)$ — different bounds, so the result depends on both

Q5

```
int[] arr = new int[n];
```

Answer: Space = $O(n)$ — array of size n

Intermediate Level

Q6

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j += 2) { ... }
```

Answer: $O(n^2)$ — inner runs $n/2$ times, drop the $1/2$ constant

Q7

```
for (int i = 1; i < n; i *= 3) { ... }
```

Answer: $O(\log n)$ — log base 3, but Big-O ignores the base

Q8

```
for (int i = 0; i < n; i++) { ... }  
for (int j = 0; j < n; j++) { ... }
```

Answer: $O(n)$ — two sequential loops add ($n + n = 2n \rightarrow O(n)$)

Q9

```
for (int i = 0; i < n; i++)  
    for (int j = i; j < n; j++) { ... }
```

Answer: $O(n^2)$ — triangle pattern, runs $n(n+1)/2$ times $\rightarrow O(n^2)$

Q10

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < i; j++)  
        for (int k = 0; k < j; k++) { ... }
```

Answer: $O(n^3)$ — three nested loops growing with each other

Advanced Thinking Level

Q11

```
int i = 0;  
while (i < n) {  
    i += 2;  
}
```

Answer: $O(n)$ — runs $n/2$ times, drop the constant

Q12

```
int i = 1;  
while (i < n) {  
    i *= 2;  
}
```

Answer: $O(\log n)$ — variable doubles each step

Q13

```
for (int i = 0; i < n; i++) {  
    i++;  
}
```

Answer: $O(n)$ — even though i increases by 2 each iteration ($++$ inside + $++$ outside), it still grows linearly with n

Q14

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < Math.sqrt(n); j++) { ... }
```

Answer: $O(n \times \sqrt{n}) = O(n^{1.5})$

Q15

```
for (int i = 0; i < n; i++) {  
    if (i % 2 == 0) { ... }  
}
```

Answer: $O(n)$ — the if just decides what happens, but the loop still runs n times

Space Complexity Practice

Q16

```
int x = 10;
```

Answer: $O(1)$

Q17

```
int[] arr = new int[n];
```

Answer: $O(n)$

Q18

```
int[][] matrix = new int[n][n];
```

Answer: $O(n^2)$

Q19

```
int[] a = new int[n];  
int[] b = new int[n];
```

Answer: $O(n)$ — $2n$ total, drop the constant

Q20

```
int[][] grid = new int[n][m];
```

Answer: $O(n \times m)$

14. Final Takeaway

If loops and arrays taught you HOW to solve problems, complexity teaches you HOW WELL to solve them.

- Time Complexity = how efficient your solution is
- Space Complexity = how smart your memory use is

The most important insight: Writing a working solution is step 1. Writing an EFFICIENT solution is step 2. Most beginners stop at step 1 and wonder why their code fails on large inputs. From now on, every solution you write should come with a complexity in your head — even if no one asks for it.

Three stages of growth as a programmer:

- Beginner — focuses on correctness (does it work?)
- Intermediate — focuses on logic (does it work cleanly?)
- Advanced — focuses on optimization (does it scale?)

Today, you are stepping into the optimization mindset. That's a big jump.

Key Takeaway: Memorize 4 things: Ignore constants. Drop lower terms. Nested loops multiply. Sequential loops add. With just these four rules, you can compute the complexity of 90% of code you'll ever encounter. The other 10% will come with experience.

15. What's Next?

With complexity solid, the natural next topics in your DSA journey are:

- Searching algorithms — Linear Search ($O(n)$) vs Binary Search ($O(\log n)$)
- Sorting algorithms — Bubble, Selection, Insertion (all $O(n^2)$) then Merge, Quick ($O(n \log n)$)
- Recursion — solving problems by breaking into smaller versions
- Recurrence relations — analyzing recursive time complexity
- Two-pointer and sliding window techniques
- Hashing for $O(1)$ lookups